

Failure Modes

 systemdesignschool.io/fundamentals/cache-failure-modes

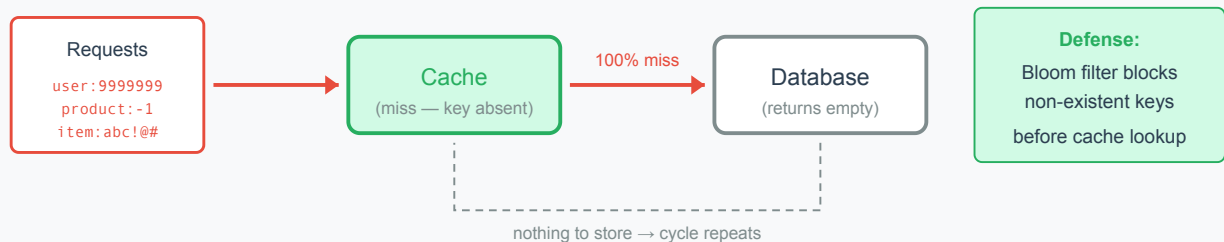


A well-configured cache typically absorbs 90–99% of read traffic, depending on access patterns and TTL strategy. The database handles the remainder. But specific failure patterns bypass the cache entirely, routing full traffic to the database. A database provisioned for that small fraction of load cannot absorb 100%. Three patterns cause this.

Cache Penetration

Penetration occurs when requests query keys that do not exist in the cache or the database. Every request is a cache miss. Every miss queries the database. The database returns empty. The cache stores nothing (there's no data to cache). The next identical request repeats the cycle.

Cache Penetration: Non-Existent Keys Bypass Cache



Every request hits the database — cache provides zero protection



An attacker or a buggy client can exploit this by requesting random non-existent IDs (`user:99999999`, `product:-1`). The cache provides no protection because these keys are never populated.

Defense 1: Cache empty results. When the database returns no data for a key, cache a sentinel value (e.g., `NULL` or an empty object) with a short TTL (30–60 seconds). Subsequent requests for the same non-existent key hit the cache and return the sentinel without touching the database.

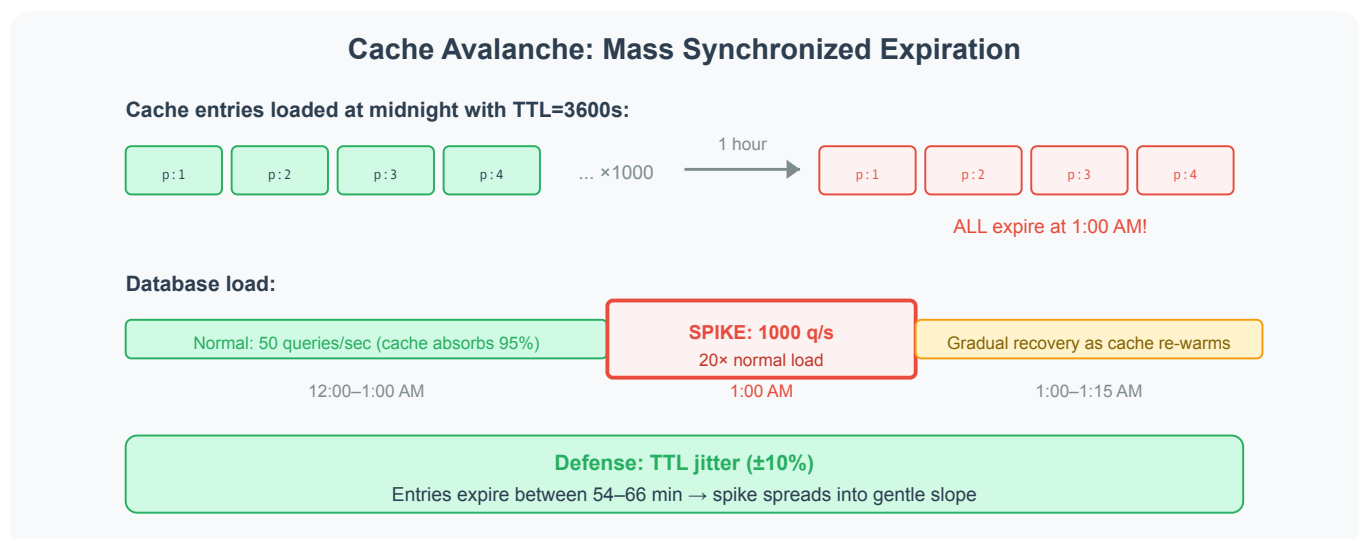
The limitation: if the attacker uses random keys (each request a different non-existent ID), the cache fills with millions of sentinel entries. Memory is wasted on entries that are never re-requested.

Defense 2: Bloom filter. A Bloom filter is a probabilistic data structure that answers "might exist" or "definitely does not exist" using minimal memory. Place a Bloom filter in front of the cache. Before checking the cache, check the Bloom filter. If the filter says "does not exist," reject the request without querying cache or database. The filter must be updated on the write path when new keys are created—otherwise legitimate new keys are incorrectly rejected.

A Bloom filter holding 100 million keys uses roughly 120 MB of memory (at 1% false positive rate). The remaining 1% of non-existent keys pass through as false positives and still hit the database, but the vast majority are blocked.

Cache Avalanche

Avalanche occurs when many cache entries expire simultaneously, causing a wave of misses that flood the database. This happens when a large batch of data is cached at the same time with the same TTL—all entries expire at the same instant.



A product catalog loaded into cache at midnight with a 1-hour TTL creates an avalanche at 1:00 AM. Every product page becomes a miss. The database receives the full catalog's read traffic at once.

Defense 1: TTL jitter. Add a random offset to each entry's TTL. Instead of all entries expiring at 3600 seconds, each gets a TTL between 3240 and 3960 seconds ($3600 \pm 10\%$). Expirations spread across 12 minutes instead of clustering at one instant.

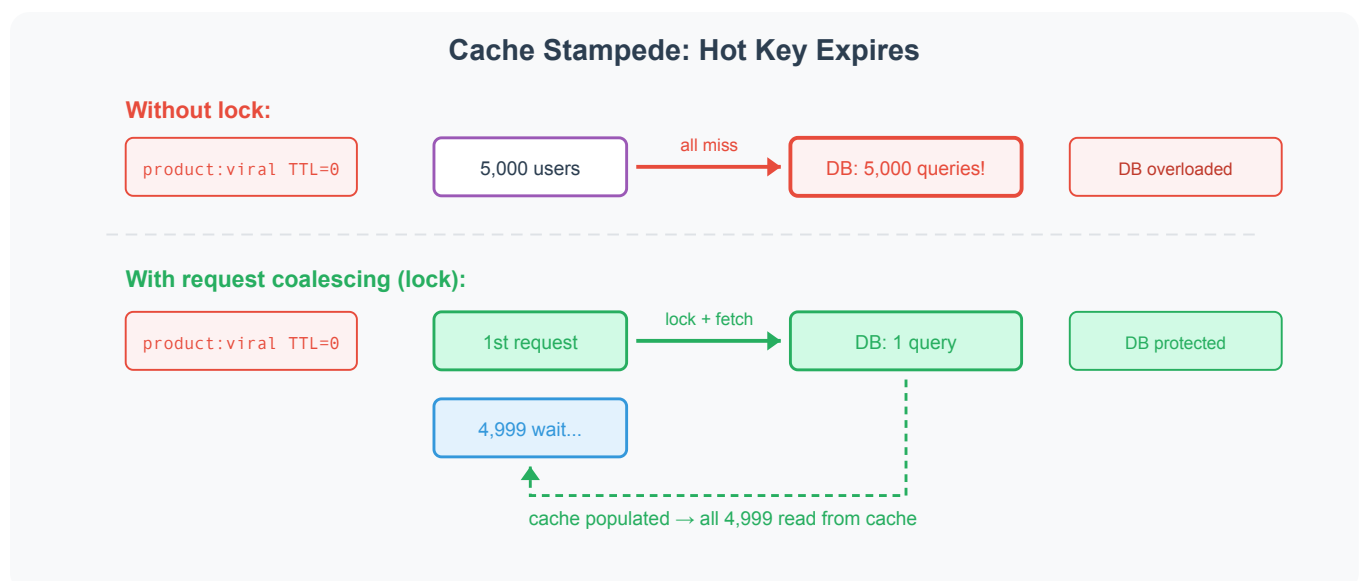
Defense 2: Staggered warm-up. After a cache restart or cold deploy, don't load all entries at once. Load entries gradually as they're requested (lazy loading). The database handles elevated but distributed load rather than a synchronized spike.

Defense 3: Multi-level caching. Use a local in-process cache (small, fast) in front of the distributed cache. Even if the distributed cache experiences an avalanche, the local cache absorbs some of the repeated hits. The database sees a reduced spike.

Cache Stampede (Thundering Herd)

Stampede occurs when a single popular key expires and many concurrent requests all experience a cache miss simultaneously. All of them query the database for the same key at the same instant.

A product page cached with a 60-second TTL expires. Within the next millisecond, 5,000 users request the same page. All 5,000 see a cache miss. All 5,000 query the database for the same row. The database executes 5,000 identical queries instead of 1.



Defense 1: Locking (request coalescing). The first request that discovers the miss acquires a lock on the key. Subsequent requests see the lock and wait (or return a stale value) instead of querying the database. Once the first request populates the cache, it releases the lock. All waiting requests read from the freshly-populated cache.

This reduces 5,000 database queries to 1. The tradeoff is added complexity: the lock must have a timeout (to prevent deadlocks if the first request fails), and waiting requests need a fallback if the lock expires.

Defense 2: Stale-while-revalidate. Return the expired (stale) value to all requests while a single background task refreshes the cache. Users get slightly stale data but never wait. This still requires coordination (a lock or singleflight mechanism) to ensure only one task triggers the refresh—without it, multiple workers may revalidate simultaneously.

Defense 3: Preemptive refresh. Refresh the cache before the TTL expires. A background job monitors popular keys and refreshes them at 80% of their TTL (e.g., refresh at 48 seconds for a 60-second TTL). Under normal operation the key is always fresh when requests arrive, so no miss occurs. If the refresh job fails or lags, the key can still expire—combine with stale-while-revalidate as a fallback.

Choosing Defenses

Failure Mode	Root Cause	Primary Defense	Fallback
Penetration	Non-existent keys	Bloom filter	Cache nulls with short TTL
Avalanche	Synchronized expiration	TTL jitter	Staggered warm-up
Stampede	Hot key expires	Request coalescing (lock)	Stale-while-revalidate

In practice, production systems deploy all three defenses simultaneously. TTL jitter is nearly free (add one random number). Bloom filters require a setup step but prevent an entire class of attacks. Request coalescing for hot keys prevents the most damaging failure mode—a single expired key taking down the database.

A hands-on lab simulates these failure modes so you can observe the impact and verify that the defenses work.